

Land Registration Management System

Overall System Flow & Architecture

1. System Overview

The Land Registration Management System is built using a **hybrid Web2 + Web3 architecture**. The goal is to combine the performance and usability of traditional web applications with the immutability, transparency, and trust guarantees provided by blockchain and decentralized storage.

Core Design Principles

- Fast querying and relational data handled off-chain
 - Critical ownership, payment, and document integrity stored on-chain
 - Immutable document storage using IPFS
 - Backend-managed blockchain interactions for non-crypto users
-

2. Architecture Overview

Hybrid Architecture Components

Web2 Layer

- **Backend:** NestJS (TypeScript)
- **Database:** PostgreSQL (TypeORM)
- **Responsibilities:**
 - User profiles and authentication
 - Application state management
 - Fast queries and relational data
 - Business logic and validations

Web3 Layer

- **Blockchain:** Ethereum-compatible EVM chain
- **Libraries:** Ethers.js
- **Responsibilities:**
 - Immutable land ownership records
 - Document hash anchoring
 - Payment verification (ERC20)
 - Reservation locking

Decentralized Storage

- **IPFS Provider:** Pinata

- **Responsibilities:**
 - Immutable storage of land documents and images
 - Verifiable content-addressed storage
-

3. Application Startup Flow

Entry Point: `main.ts → app.module.ts`

Initialization Steps

1. Bootstrap NestJS application
 2. Load environment configuration (`.env`)
 3. Initialize PostgreSQL connection using TypeORM
 4. Initialize blockchain provider (RPC + Ethers.js)
 5. Initialize smart contract instances
 6. Configure global application features:
 7. Validation pipes (`class-validator`)
 8. Global exception filters
 9. Response transform interceptors
 10. CORS (allow all origins)
 11. Static file serving (`uploads/`)
 12. Swagger documentation (`/api/docs`)
-

4. Authentication & Authorization Flow

Registration Flow

Endpoint: `POST /api/auth/register`

1. Validate email uniqueness
2. Hash password using bcrypt
3. Create user record in database
4. Generate HD wallet address automatically
5. Master mnemonic-based derivation
6. Path: `m/44'/60'/0'/0/{userIndex}`
7. Store wallet address in user record
8. Generate JWT access and refresh tokens
9. Return user profile and tokens

Login Flow

Endpoint: `POST /api/auth/login`

1. Fetch user by email

2. Verify password hash
3. Auto-generate wallet if missing
4. Generate JWT access and refresh tokens
5. Return user profile and tokens

Authentication Middleware

- JWT validation via `JwtAuthGuard`
 - User context injected from token payload
 - Role-based access control via `RolesGuard`
-

5. Land Listing Flow (`lands.module`)

Create Land Listing

Endpoint: `POST /api/lands` (multipart/form-data)

1. Seller/Admin submits land data with files
2. Uploaded files:
3. Legal document (deed/contract)
4. Property image
5. Files saved to local storage (`uploads/`)
6. SHA-256 hash calculated for each file
7. Files uploaded to IPFS (Pinata)
8. Blockchain registration:
9. Call `registerLand()` smart contract function
10. Parameters: owner address, IPFS hash, document hash, price
11. Receive blockchain land ID and transaction hash
12. Persist land record in database:
13. Local file paths
14. IPFS hashes
15. SHA-256 hashes
16. Blockchain land ID
17. Blockchain transaction hash
18. Land status set to `AVAILABLE`

Document Verification

Endpoint: `GET /api/lands/:id/verify`

1. Load file from local storage
 2. Recalculate SHA-256 hash
 3. Compare against stored hash
 4. If matched, document integrity is confirmed
 5. Optional on-chain hash verification
-

6. Reservation Flow (reservations.module)

Create Reservation

Endpoint: POST /api/reservations

1. Validate:
2. Land exists
3. Land status is AVAILABLE
4. Buyer has wallet address
5. No active reservation exists
6. Blockchain operation:
7. Call lockLandToBuyer()
8. Lock land to buyer wallet address
9. Database updates:
10. Land status → LOCKED
11. Create reservation record (ACTIVE)
12. Prevents other users from reserving the land

Cancel Reservation

Endpoint: DELETE /api/reservations/:id

1. Update reservation status → CANCELLED
 2. If no other active reservations:
 3. Land status → AVAILABLE
-

7. Payment Flow (payments.module)

Create Payment

Endpoint: POST /api/payments

Crypto Payment

1. Retrieve buyer private key from HD wallet
2. Approve ERC20 tokens if required
3. Call makeERC20Payment() on smart contract
4. Tokens transferred from buyer to contract
5. Store blockchain transaction hash

Bank Payment

1. Upload payment proof (receipt/screenshot)
2. Store proof file locally

3. Create payment record with status `PENDING`

4. Notify seller for verification

Verify Payment

Endpoint: `POST /api/payments/:id/verify`

1. Validate seller ownership of land
 2. Update payment status:
 3. `VERIFIED` or `REJECTED`
 4. If verified:
 5. Check all required payments
 6. Update land status → `SOLD`
 7. Future enhancement:
 8. On-chain verification via `verifyBankPayment()`
-

8. Blockchain Integration Layer

Backend-Managed Wallet Model

- Admin private key stored securely in environment
- Backend signs transactions on behalf of users
- No MetaMask or crypto knowledge required
- Optimized UX for non-crypto users

Smart Contract Operations

- `registerLand()`
 - `lockLandToBuyer()`
 - `updateLand()`
 - `makeERC20Payment()`
 - `verifyBankPayment()`
 - `getPaymentBreakdown()`
-

9. HD Wallet System (`wallet.service.ts`)

Master Wallet

- Single master mnemonic (`MASTER_WALLET_MNEMONIC`)
- Stored securely in environment

User Wallet Derivation

1. User UUID converted to numeric index

2. Derivation path: `m/44'/60'/0'/0/{index}`
3. Deterministic wallet generation
4. Private key derived for transaction signing
5. Wallet address stored in database

Benefits

- No seed phrase management for users
 - Automatic wallet recovery
 - Unique Ethereum address per user
 - Backend-controlled signing
-

10. Data Flow Diagram

Client → REST API → NestJS Backend

- Auth Module → JWT / RBAC Guards
- Business Services:
 - Lands Service
 - Reservations Service
 - Payments Service

Backend Integrations: - PostgreSQL (relational data) - Blockchain (EVM smart contracts) - IPFS (Pinata)

11. Security Features

- Bcrypt password hashing
 - JWT authentication (access & refresh tokens)
 - Role-based access control
 - File integrity verification (SHA-256)
 - SQL injection prevention via TypeORM
 - Input validation using `class-validator`
-

12. Transaction Lifecycle (End-to-End)

1. **Registration**
2. User registers
3. Wallet generated
4. **Land Listing**
5. Seller uploads documents

6. IPFS storage

7. Blockchain registration

8. Reservation

9. Buyer reserves land

10. Blockchain lock

11. Payment

12. Bank proof or ERC20 transfer

13. Payment marked PENDING

14. Verification

15. Seller verifies payment

16. Land marked SOLD

17. Ownership Transfer (Future)

18. Final on-chain ownership update

13. Technology Stack Summary

- **Framework:** NestJS (TypeScript)
 - **Database:** PostgreSQL (TypeORM)
 - **Blockchain:** Ethereum (Ethers.js)
 - **Smart Contracts:** Solidity (UUPS Upgradeable)
 - **Storage:** Local + IPFS (Pinata)
 - **Authentication:** JWT (Passport)
 - **Email:** Nodemailer (Gmail SMTP)
 - **API Documentation:** Swagger / OpenAPI
-

This system delivers high performance, security, and transparency by combining traditional backend architecture with blockchain immutability and decentralized storage.